

Implementing Neural Networks in TensorFlow

TensorFlow is one of the most popular open-source frameworks used to build and train deep learning models. This course will focus on TensorFlow for its implementations (an alternative framework is PyTorch).

We will use TensorFlow to implement **inference** (the process of making a prediction) by defining the layers of our network.

The Dense Layer

In TensorFlow, the standard, fully-connected layer we have been studying is called a **Dense** layer.

- **Definition:** A Dense layer means that every neuron in that layer is connected to every activation from the previous layer.
- **Syntax:** You define a layer using `Dense(units=..., activation=...)`
 - **units:** The number of neurons in the layer.
 - **activation:** The activation function to be used by the neurons in that layer (e.g., 'sigmoid').

Forward Propagation in TensorFlow

The process of feeding an input x through the layers to compute the final output is called **forward propagation**.

Example 1: Coffee Roasting (2-layer network)

- **Goal:** Predict if coffee is good (1) or bad (0) based on 2 features.
- **Input (Layer 0):** $\vec{x} = [200, 17]$ (temperature, duration)

```
1 # Layer 1 (Hidden Layer)
2 # Define the layer's structure (3 neurons, sigmoid)
3 Layer1 = Dense(units=3, activation='sigmoid')
4 # Compute the layer's output (forward prop)
5 a1 = Layer1(x) # a1 will be a vector with 3 activations
6
7 # Layer 2 (Output Layer)
8 # Define the layer's structure (1 neuron, sigmoid)
9 Layer2 = Dense(units=1, activation='sigmoid')
10 # Compute the final output
11 a2 = Layer2(a1) # a2 is the final probability (e.g., 0.8)
12
13 # Optional: Make a binary prediction
```

```
14 | if a2 >= 0.5:
15 |     y_hat = 1
16 | else:
17 |     y_hat = 0
```

Example 2: Handwritten Digits (3-layer network)

This example shows how to stack multiple layers.

- **Input (Layer 0):** $\vec{x}$ (a vector of pixel intensity values)

```
1 | # Layer 1 (Hidden)
2 | Layer1 = Dense(units=25, activation='sigmoid')
3 | a1 = Layer1(x)
4 |
5 | # Layer 2 (Hidden)
6 | Layer2 = Dense(units=15, activation='sigmoid')
7 | a2 = Layer2(a1)
8 |
9 | # Layer 3 (Output)
10 | Layer3 = Dense(units=1, activation='sigmoid')
11 | a3 = Layer3(a2) # a3 is the final prediction
```

Note: The lab exercises will provide the code for loading the TensorFlow library and the pre-trained parameters (w and b) for each layer. The code above defines the *architecture* and executes the *forward propagation* steps.

Data Representation in NumPy and TensorFlow

The Core Inconsistency

- **Historical Context:** The Python data science ecosystem evolved in stages. **NumPy** was created first as the standard for numerical operations. **TensorFlow** was created much later and, while it interoperates with NumPy, it has its own internal conventions for efficiency.
 - **Key Takeaway:** You must be aware of the different data structures (especially 1D vs. 2D arrays) to ensure your code works correctly.
-

Matrices (2D Arrays)

- A **matrix** is a 2D array of numbers.
- The dimension of a matrix is always specified as **Rows x Columns**.
- **Example (2x3 Matrix):**
 - This matrix has 2 rows and 3 columns.

```
1 # [[1, 2, 3],
2 #  [4, 5, 6]]
3 x = np.array([[1, 2, 3], [4, 5, 6]])
```

The Key Difference: 1D vs. 2D Arrays (Vectors)

This is the most important convention to understand for this course.

- **Course 1 (NumPy Standard): 1D Array**
 - We represented a feature vector with a *single* set of square brackets.
 - `x = np.array([200, 17])`
 - This creates a **1D array**. It is just a list and technically has **no rows or columns**. Its shape is simply `(2, )`.
- **Course 2 (TensorFlow Convention): 2D Array**
 - TensorFlow is designed to process data in "batches" (matrices). To represent a *single* training example, we must use a **2D array** (a matrix).
 - We use *double* square brackets to create a **row vector**.
 - `x = np.array([[200, 17]])`
 - This creates a **1x2 matrix** (1 row, 2 columns). This is the standard way to feed a single instance into a TensorFlow model.

Code	Type	Dimensions	Shape
<code>np.array([200, 17])</code>	1D Array (1D Vector)	None	(2,)
<code>np.array([[200, 17]])</code>	2D Array (Row Vector)	1 row, 2 cols	(1, 2)
<code>np.array([[200], [17]])</code>	2D Array (Column Vector)	2 rows, 1 col	(2, 1)

The TensorFlow tensor

- When you pass a NumPy array into a TensorFlow layer, TensorFlow converts it into its own internal data type called a **tf.tensor**.
- A **tensor** is just TensorFlow's optimized way of representing a matrix (or vector, or higher-dimensional data).
- For the purposes of this course, you can think of a **tensor** as being equivalent to a **matrix**.

Data Shapes During Inference

When you pass data through your network, the shape of the tensor will change based on the layers.

1. **Input x:** You pass in a **1x2 matrix** (a tensor with shape (1, 2)).
 - `x = np.array([[200, 17]])`
2. **Layer 1 (3 units):** `a1 = Layer1(x)`
 - The layer takes the 1x2 input and computes 3 activations.
 - The output `a1` will be a **1x3 matrix** (a tensor with shape (1, 3)).
3. **Layer 2 (1 unit):** `a2 = Layer2(a1)`
 - The layer takes the 1x3 input and computes 1 activation.
 - The output `a2` will be a **1x1 matrix** (a tensor with shape (1, 1)).

Converting Data

You can easily convert between TensorFlow Tensors and NumPy arrays.

- **Tensor** → **NumPy:** Use the `.numpy()` method.
 - `a1_numpy = a1.numpy()`

Building a Neural Network in TensorFlow

While you can compute the output of a network one layer at a time, TensorFlow provides a much simpler and more powerful way to build, train, and run models using the **Sequential API**.

The Sequential Model: A Simpler Approach

Instead of manually passing the output of one layer to the input of the next, the `Sequential` model allows you to define a "stack" or sequence of layers. TensorFlow then handles the work of stringing them together.

- **Old Method (Manual Forward Prop):**

```
1 a1 = Layer1(x)
2 a2 = Layer2(a1)
3 a3 = Layer3(a2)
```

- **New Method (Using `Sequential`):**

You define all the layers at once, and TensorFlow connects them for you.

A Common Coding Convention 🏠

The most common way to build a model is to define the layers directly *inside* the `Sequential` function.

- **Example (Digit Classification):**

```
1 import tensorflow as tf
2 from tensorflow.keras.layers import Dense
3 from tensorflow.keras.models import Sequential
4
5 model = Sequential([
6     Dense(units=25, activation='sigmoid'), # Layer 1
7     Dense(units=15, activation='sigmoid'), # Layer 2
8     Dense(units=1, activation='sigmoid')  # Layer 3 (Output)
9 ])
```

- This single `model` object now contains the entire three-layer network architecture.

How to Use the Sequential Model

Once you have a `Sequential` model, training it and making predictions becomes simple.

- **1. Training the Model (Preview):**

- `model.compile(...)`: A required setup step to configure the model for training (details will be covered next week).
- `model.fit(X, Y)`: A single command to **train** the network. You provide the training data (`X`) and the target labels (`Y`), and TensorFlow handles the entire gradient descent process.

- **2. Making Predictions (Inference):**

- `model.predict(X_new)`: This is the new way to perform forward propagation. You pass in a new input (`X_new`), and the model automatically runs it through all the layers and returns the final prediction.
-

Understanding "Under the Hood"

- While frameworks like TensorFlow allow you to build complex neural networks in just a few lines of code, it is critical to understand *what* those lines are actually doing.
- Most engineers use libraries, but the best engineers understand the underlying mechanics. To build this understanding, the next step is to learn how to implement forward propagation from scratch.